



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Precision, speed, error growth

Project Report

Author:

Haoanqin Gao
Jonas Hermann
Rochelle Leung

Supervisor:

Max Harvey

701-1270-00L High Performance Computing for Weather and Climate

Institute for Atmospheric and Climate Science
Department of Environmental Systems Science, ETH Zürich

August 31, 2026

1. Introduction

Conventionally, 64-bit double precision is used for all components in computational fluid dynamics modeling, including weather and climate modeling (e.g. Banderier et al., 2024; Kimpson et al., 2023; Hatfield et al., 2018; Rüdisühli et al., 2014), as it was believed that the chaotic nature of the non-linear equations (Lorenz, 1963) may cause small rounding errors to build up. In addition, the use of high precision could prevent linear algebra solvers from non-convergence and ensure that the solutions are up to the desired degree of accuracy (Brogi et al., 2024). However, the increase in numerical precision would lead to a reduction in floating point operation rates and an increase in memory requirement, so less data can be fit into the cache or communicated among processors. Under the consideration of computational cost, in recent years, researchers started to question if such high precision is necessary, or if computational resources should be relocated to other aspect, such as the increase in model resolution, as long as the error is acceptable. The use of reduced precision has been tested in various components of atmospheric modeling, including data assimilation (Hatfield et al., 2018), parameterization (Saffin et al., 2020), numerical weather prediction (Rüdisühli et al., 2014), and climate simulation (Banderier et al., 2024).

To investigate the trade-off between numerical precision and resolution in the consideration of computational speed and performance, this project performs numerical experiments at various floating point precisions, from the 16-bit half-precision (fp16), 32-bit single-precision (fp32), to the 64-bit double precision (fp64), and at various model resolutions, using the two-dimensional Burger’s equation as an example. As a simplified form of the full Navier-Stokes equation, the Burger’s equation described only the interaction between convection and diffusion for incompressible viscous flow. It captures the shock wave phenomenon for which the non-linear term steepens wave fronts while the viscous term prevents actual discontinuities at large Reynold numbers, which mimics the frontogenesis in actual weather (Gao and Zou, 2017). This makes the Burger’s equation an ideal testing ground for our experiments. In this report, the methodology of the experiments are presented in Section 2, and the results and discussion are provided in Section 3. The result would facilitate our understanding on to what extent the numerical precision is important for weather and climate modeling.

2. Model

To carry out numerical experiments at different precisions and resolutions, a base script was first built to implement the two-dimensional Burger’s equation in three-dimensional space, with the numerical scheme, initial condition, boundary condition, and computational specifications described below.

2.1 Numerical scheme

In this project, for simplicity, the explicit forward-in-time scheme, upwind scheme, and central differencing scheme were used to compute the first-order time derivatives, first-order spatial derivatives and second-order spatial derivatives, respectively. Considering a

2 Model

function $u(x)$, Taylor expansion at $x = x_0$ gives

$$u(x) = u(x_0) + \left. \frac{\partial u}{\partial x} \right|_{x_0} (x - x_0) + \frac{1}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{x_0} (x - x_0)^2 + \dots \quad (2.1)$$

Rearranging, the first-order derivative at x_0 is

$$\left. \frac{\partial u}{\partial x} \right|_{x_0} = \frac{u(x) - u(x_0)}{x - x_0} + \frac{x - x_0}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{x_0} + \dots \quad (2.2)$$

Discretization gives

$$\left. \frac{\partial u}{\partial x} \right|_{i,j,k}^t = \frac{u_{i,j,k}^t - u_{i-1,j,k}^t}{\Delta x} + \frac{\Delta x}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j,k}^t + O(\Delta x^2) \quad \text{or} \quad (2.3)$$

$$\left. \frac{\partial u}{\partial x} \right|_{i,j,k}^t = \frac{u_{i+1,j,k}^t - u_{i,j,k}^t}{\Delta x} - \frac{\Delta x}{2} \left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j,k}^t + O(\Delta x^2) \quad (2.4)$$

These two equations are the formulas for the forward and backward differencing respectively. When adopting Equation 2.3 for wind speed $u > 0$ and Equation 2.4 for $u < 0$ with the higher-order terms dropped, this resembles the upwind scheme for computing the first-order spatial derivatives, which is conditionally stable with appropriate time steps. When using Equation 2.3 alone, the explicit forward-in-time scheme to compute the time derivatives is obtained. Finally, taking the difference between Equation 2.3 and Equation 2.4, the formula to compute the second order derivatives can also be derived.

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{i,j,k}^t = \frac{u_{i+1,j,k}^t - 2u_{i,j,k}^t + u_{i-1,j,k}^t}{\Delta x^2} + O(\Delta x^2) \quad (2.5)$$

The above numerical schemes require one halo point in space for computing derivatives at the boundaries.

2.2 Boundary condition

To avoid the presence of artificial boundaries within the model domain, the periodic boundary condition is used. This limits the causes of any error growth to either the choice of numerical schemes, precision, and resolution, and is also physically relevant for modeling the spherical Earth with no horizontal boundaries.

2.3 Initial condition

In research, that wavy initial condition that combines sine and cosine is frequently used to study the shock wave phenomenon (Gao and Zou, 2017). Since our project adopts the periodic boundary condition, this initial condition can be adapted by projecting the full 2π wavelength to the model domain, that is,

$$u(x, y, z) = \sin(2\pi x) \cos(2\pi y) \quad (2.6)$$

$$v(x, y, z) = \cos(2\pi x) \sin(2\pi y) \quad (2.7)$$

for x , y , and z from 0 to 1. The leftmost panel of Figure 3.1 illustrates the initial u field at resolution 4096×4096 and fp32 precision.

2.4 Computational specifications

The above implementation was coded in C and parallelized with OpenMP (Dagum and Menon, 1998) to ran in parallel over the 72 CPU cores on the Alps supercomputer. The model runs from time $t = 0$ to $t = 1$ in a fixed timestep of

$$\Delta t = \min \left(\frac{1}{2} \cdot \frac{1}{V_{\max}/\Delta x + V_{\max}/\Delta y}, \frac{1}{2} \cdot \frac{\text{Re}}{1/\Delta x^2 + 1/\Delta y^2} \right)$$

except at the designated times for output. The first argument of $\min()$ is the Courant–Friedrichs–Lewy (CFL) condition for advection and the second argument the diffusive restriction. We set the Reynolds number to 10'000 so that for the resolutions we are working with, the advective term dominates. The parallelization is done with static scheduling and collapse over the j and k indexes.

3. Results and discussion

The analysis for the speed and performance of different experimented configurations are presented in Section 3.1, while the results in terms of precision and error are reported in Section 3.2. Lastly, based on the results, in Section 3.3, we propose an optimized approach to achieve the best accuracy in a fixed amount of time. To provide an idea of how the simulation output looks like, Figure 3.1 shows an example with several snapshots of the u field at resolution 4096×4096 and fp32 precision.

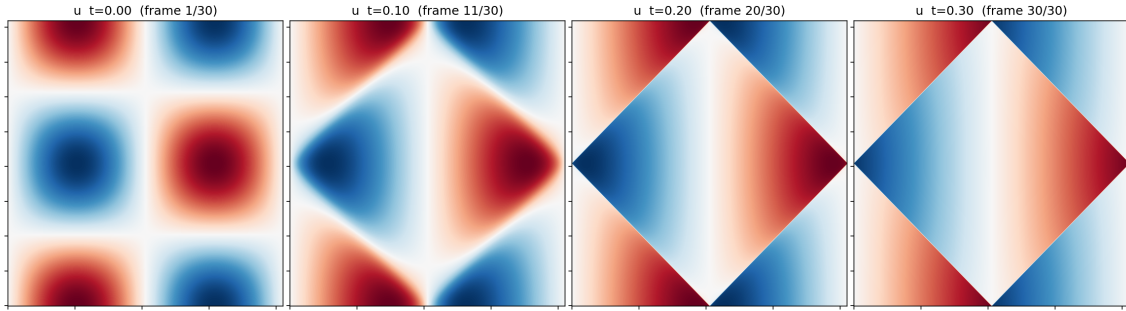


Figure 3.1: Time evolution of the u field at resolution 4096×4096 and fp32 precision. From left to right are the snapshots at $t = 0.0$, $t = 0.1$, $t = 0.2$, $t = 0.3$.

3.1 Speed and performance

To inspect the impact on simulation speed by using floating-point numbers with different precisions and the corresponding scaling with mesh resolution, we run the simulation using the same initial condition for fp16/32/64 and mesh resolution ranging from 16×16 to 8192×8192 on the XY-plane. The Z-dimension is fixed at 10 layers since it has no

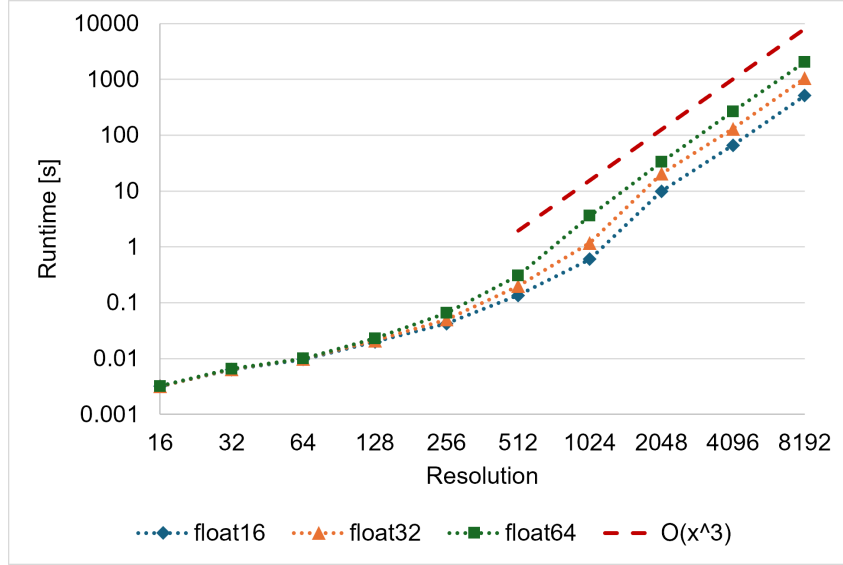


Figure 3.2: Benchmark runtime of the stencil computation for resolutions ranging from 16×16 to 8192×8192 using three different floating-point precisions. The red dashed line is drawn as a reference for $\mathcal{O}(x^3)$ growth.

86 impact on the two dimensional governing equation. Note that because we impose the CFL
87 condition with a fixed maximum velocity, the number of time steps for a given resolution
88 scales with $\mathcal{O}(\Delta x^{-1})$, and the amount of computation scales with $\mathcal{O}(\Delta x^{-2})$. Therefore,
89 we expect the total runtime to scale with $\mathcal{O}(\Delta x^{-3})$ for square XY-planes.

90 For each precision-resolution pair, we make multiple runs and take the average time so
91 that at least 10 wall clock seconds elapse and a more reliable timing result is recorded. In
92 the runtime log-log plot (Figure 3.2), there is a visible jump for the fp64 line going from
93 resolution 512×512 to 1024×1024 for the XY-plane. Similar jumps are also present for
94 fp32 and fp16 going from resolution 1024×1024 to 2048×2048 . These jumps can be
95 explained by looking at the memory consumption of the respective stencils and comparing
96 it to the total cache size of the Grace CPU (Table 3.1). Because we need to store the u
97 and v field at the current and previous time step, the total memory consumption is given
98 by

$$\text{memory consumption} = 4 \cdot \text{\#layer} \cdot \text{layer resolution} \cdot 1/2/4 \text{ bytes.}$$

99 The Grace CPU (72 Arm Neoverse V2 cores) has for each core 64 KiB L1 data cache
100 and 1 MiB L2 cache, and 114 MiB shared L3 cache, giving us a total of 191 MiB cached
101 memory thanks to its non-inclusive cache architecture (data stored on one cache level do
102 not have to also exist on other levels). Comparing this to the memory consumption of the
103 precision-resolution pairs, we see that for fp16 and fp32, resolutions below and including
104 1024×1024 can fit inside the cache, while for fp64, only 512×512 and below are able to
105 fit. This explains well the steep increase in runtime, as retrieving data from main memory
106 is much slower than retrieving from cache.

107 At resolution 2048×2048 and above, we see consistent timing results for all three pre-
108 cisions, comparing the slope versus that of $\mathcal{O}(x^3)$ shows that the runtime indeed increases
109 cubically (not simply quadratically due to the additional time steps taken for higher res-

3 Results and discussion

Resolution ($10 \times n \times n$)	fp16	fp32	fp64
16	2.0×10^4	4.1×10^4	8.2×10^4
32	8.2×10^4	1.6×10^5	3.3×10^5
64	3.3×10^5	6.6×10^5	1.3×10^6
128	1.3×10^6	2.6×10^6	5.2×10^6
256	5.2×10^6	1.0×10^7	2.1×10^7
512	2.1×10^7	4.2×10^7	8.4×10^7
1024	8.4×10^7	1.7×10^8	3.4×10^8
2048	3.4×10^8	6.7×10^8	1.3×10^9
4096	1.3×10^9	2.7×10^9	5.4×10^9
8192	5.4×10^9	1.1×10^{10}	2.1×10^{10}

Table 3.1: Memory consumption of all precision-resolution pairs in bytes, compared with the total cache size of the Grace CPU (2.0×10^8 bytes), sizes smaller than the cache size are colored green, sizes larger are colored orange.

olutions, as imposed by the CFL condition).

Now, we turn our attention to the performance of our simulation. Counting the number of floating-point operations in our core Burgers kernel, for every element of the stencil, 44 addition or multiplication Floating Point Operations (FLOPs) are executed. This gives a total FLOP count of

$$\#FLOP = 44 \cdot \#time\ step \cdot \#layer \cdot layer\ resolution$$

with the number of time steps being the total time $t_{total} = 1$ divided by Δt . For the case of $\Delta x = \Delta y$ and the maximum velocity being 1, $\#time\ step$ simplifies to $4/\Delta x$. Because we also output the u and v fields at 0.01 time interval, the actual number of time steps is rounded up to the next hundred. Using the FLOP counts, we can then compute the performance of our code for different resolutions and compare it with the theoretical peak performance of the Grace CPU, which is documented for various float-point precisions.

As seen in Figure 3.3, for very low resolutions, even though everything fits in the cache, we do not properly utilize the CPU because there simply is not enough work to fill the pipeline and saturate parallelism, the initialization/cleanup overhead is also high relative to the actual work. The performance climbs as we increase the resolution, up to 22.3% of the theoretical peak performance for fp64 at 512×512 , and 22.7% and 22.1% for fp32 and fp16 respectively at 1024×1024 . These resolutions match the maximum resolution for each precision where all the data still fit within the cache (see Table 3.1). As we continue to increase the resolution, the data now have to be stored and retrieved from the memory. This bottlenecks the CPU. We see that for all three precisions, the performance stabilizes at around 13.2% of the theoretical peak performance, equating to 0.48 TFLOPS for fp64, 0.93 TFLOPS for fp32, and 1.88 TFLOPS for fp16.

To better assess whether our final performance is adequate, we turn to the roofline model (Ofenbeck et al., 2014) for the Grace CPU (Figure 3.4), three theoretical peak performance lines are drawn for the three floating-point precisions in our experiment, together with the memory-bound performance computed from the system’s 512 GB/s memory bandwidth. As we see from the plot, to achieve the peak performance, an operational intensity of 13.9, 27.7, and 55.5 FLOP/byte is needed for fp64, fp32, and fp16,

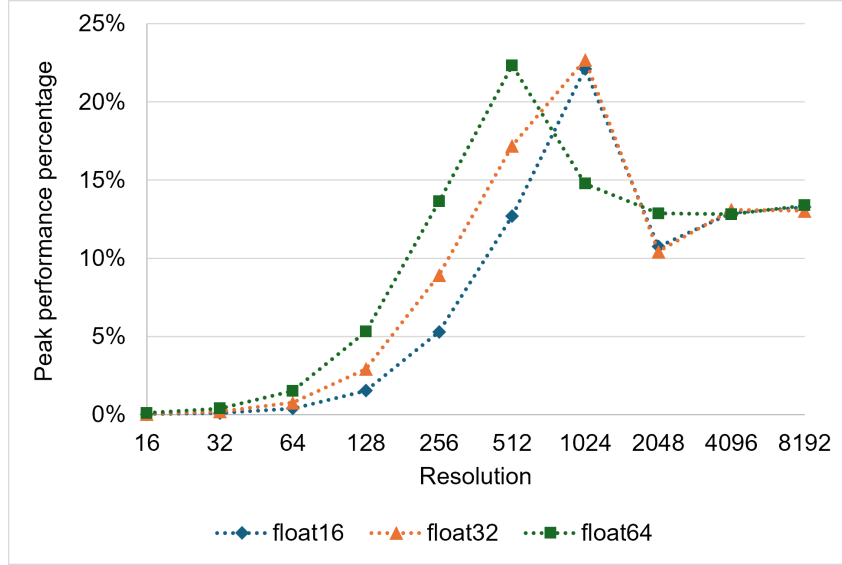


Figure 3.3: Performance as a percentage of the theoretical peak floating-point performance of the Grace CPU for resolutions ranging from 16×16 to 8192×8192 , note that the peak performance differs for different floating-point precisions (fp16: 14 TFLOPS, fp32: 7.1 TFLOPS, fp64: 3.6 TFLOPS).

138 respectively. However, since we require four floating-point numbers for the 44 FLOPs, as
 139 counted earlier, we get an actual operational intensity of 1.4, 2.8, and 5.5 FLOP/byte.
 140 Therefore, it can be concluded that we are strictly in the memory-bound region and the
 141 maximum performance achievable is 0.70 TFLOPS for fp64, 1.41 TFLOPS for fp32, and
 142 2.82 TFLOPS for fp16. Thus, we have achieved on average 66.7% of the maximum pos-
 143 sible performance using only OpenMP directives. Further optimization can be performed
 144 by, for example, improving the memory access pattern through blocking for L2 and L3
 145 cache to reduce non-compulsory cache misses.

146 3.2 Precision and error

147 Now, we examine the error incurred and the feasibility of using lower precision floating-
 148 point representations for Burgers equation, again using the sinusoidal initial condition
 149 with periodic boundary. Due to the limitation of storage capacity on Alps, we restrict
 150 the scope of our experiment to resolutions ranging from 16×16 to 4096×4096 and only
 151 simulate until $t = 0.3$, as the standing shock already fully develops at around $t = 0.2$ and
 152 only diffusion plays a role afterwards.

153 The first thing we notice when running the experiment is the fact that fp16 fails and
 154 produces invalid results at resolutions higher than 128×128 . Inspecting the output for
 155 resolution 256×256 , we see that as the standing shock develops, NaN values are produced
 156 and spread from the corners and centers of the four shock fronts. Such onset of NaN occurs
 157 earlier at higher resolutions. This can be explained by, as the resolution increases, the
 158 shock becomes sharper and the corresponding computed gradient becomes larger, leading
 159 to values that cannot be represented by fp16. Switching to higher precisions of fp32 and
 160 fp64 resolves this issue and produces valid results up to the maximum resolution tested.

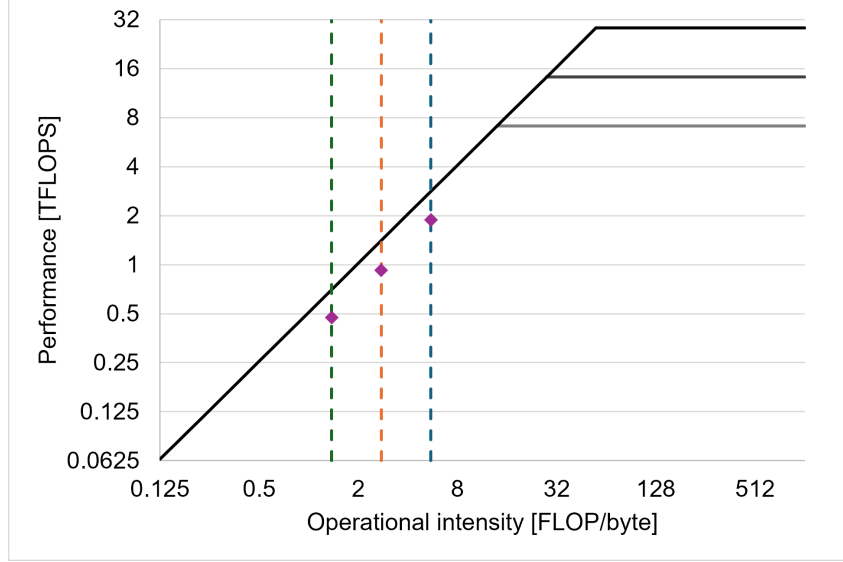


Figure 3.4: Roofline model of the Grace CPU. Peak performance lines of fp16, fp32, fp64 are drawn from top to bottom. The colored dashed lines from left to right mark the operational intensity of the computation using fp64, fp32, fp16, with the corresponding achieved performance marked by the purple diamond.

Resolution ($10 \times n \times n$)	fp16	fp32	fp64
16	18.92%	18.83%	18.83%
32	12.65%	12.49%	12.49%
64	9.10%	8.60%	8.60%
128	7.57%	5.96%	5.96%
256	-	3.97%	4.03%
512	-	2.47%	2.59%
1024	-	1.38%	1.45%
2048	-	0.56%	0.57%
4096	-	0.00%	REF

Table 3.2: The root-mean-square(RMS) error of all precision-resolution pairs relative to the maximum absolute value of the reference u field.

3 Results and discussion

Next, we take the output field with the highest precision (fp64) and resolution (4096×4096) at $t = 0.24$ (right after the shock fully develops) as a reference and compare it with all other precision-resolution pairs. To compute the error, we downsample the reference field to the target field resolution by taking the average of each $b \times b$ blocks, with b being the integer ratio between the reference and target resolution, then we compute the root-mean-square (RMS) difference between the two fields and scale the difference with respect to the maximum absolute u/v value of the downsampled reference field. The results are listed in Table 3.2.

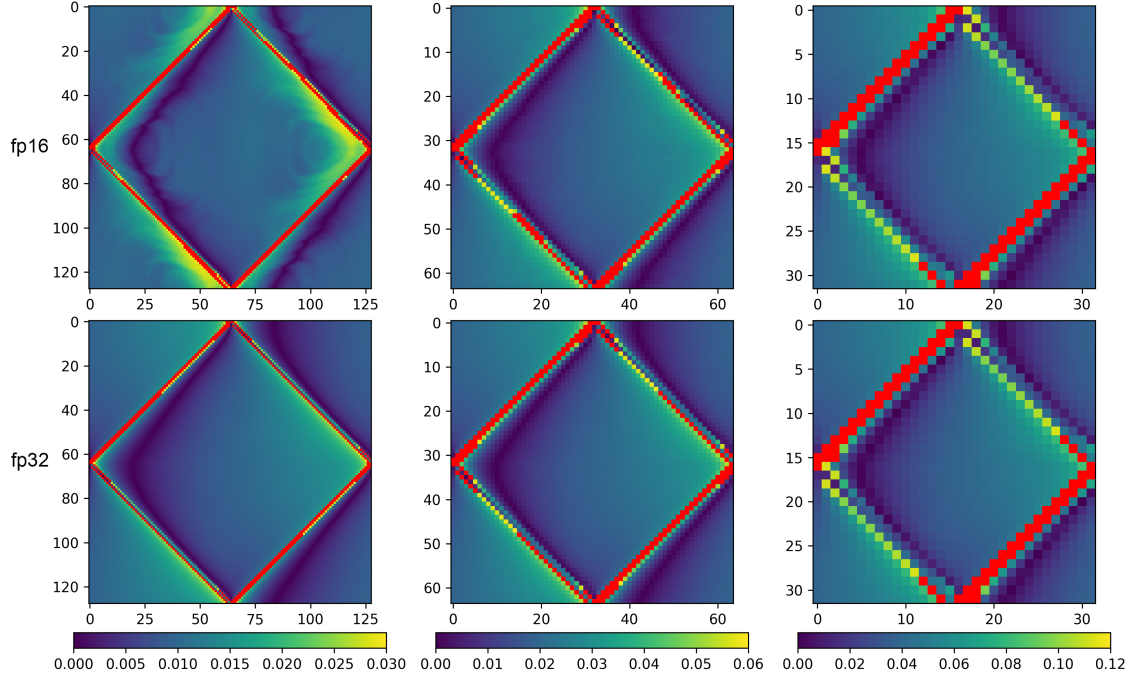


Figure 3.5: The u field at $t = 0.24$ computed at three resolutions (from left to right: 128×128 , 64×64 , 32×32) using fp16 and fp32, compared against the reference field computed at 4096×4096 using fp64, the RMS errors are plotted. To better show the ripple pattern the color scale is shrunk and errors over the scale are colored red.

There is a significant improvement to the accuracy when we switch from fp16 to fp32, not only because we can increase the resolution, but also at lower resolutions there is a non negligible difference, especially at 128×128 . Inspecting the errors at resolution 128×128 (Figure 3.5), we see that while the error field for fp32 is smooth, the field for fp16 has an additional ripple pattern "laid over" the same base field. Decreasing the resolution, we see that the ripple pattern is gone from fp16 and the two plots look virtually identical. Given that the ripples are present only at the highest possible resolution for fp16 and are not near the shocks, we suspect this is caused by catastrophic cancellation: two near identical floating-point numbers are subtracted, due to the low number of mantissa of fp16 the remaining part is mostly noise, which then gets amplified when we divide it by a tiny Δx . As to why we see the ripple pattern rather than simply a noisy error field, we think the error might periodically originate from the center of the stencil where the gradient is the smallest and most prone to cancellation, it then gets advected toward the shocks.

From Table 3.2, we also see that, across the range of tested resolutions, there is no difference in terms of accuracy between fp32 and fp64. In fact, in some cases, fp32 has

184 slightly less RMS error than fp64 for unknown reasons. But if we were to test with even
 185 higher resolution, we expect to see for fp32 the same situation as with fp16 where it no
 186 longer has the precision to resolve the steeper gradients, and we would inevitably have to
 187 switch to fp64.

188 3.3 Optimizing for accuracy

189 From the findings of Sections 3.1 and 3.2 we theorize that we should use the lowest
 190 permissible precision that still produces valid results while not hitting the threshold where
 191 the effect of cancellation becomes significant, and use the gain in performance to increase
 192 the resolution, thus achieving a more accurate simulation. As going from fp32 to fp64
 193 halves the performance on the Grace CPU and as the runtime scales with $\mathcal{O}(\Delta x^{-3})$,
 194 we can equate the runtime of computing with fp64 at a certain resolution with that of
 195 computing with fp32 at a higher resolution

$$2 \cdot \left(\frac{1}{\Delta x}\right)^3 = \left(\frac{1}{c \cdot \Delta x}\right)^3$$

196 and solve for c :

$$c = 2^{-\frac{1}{3}} \approx 0.79,$$

197 so solving the two-dimensional Burgers equation with fp32 at $0.79\Delta x$ should take the
 198 same amount of time as solving with fp64 at Δx while providing a more accurate solution.
 199 To verify this, we run two configurations with our solver, one with fp64 at resolution
 200 1024×1024 , the other with fp32 at 1290×1290 , both using 20 layers instead of 10 to
 201 make sure we are well within the memory-bound performance regime, and compare the
 202 results with the reference solution computed at 4096×4096 . We make six runs for each
 203 configuration and take the best time:

- 204 • fp32 @ 1290×1290 takes 2.21 seconds and achieves an RMS error of 1.04%,
- 205 • fp64 @ 1024×1024 takes 2.11 seconds and achieves an RMS error of 1.46%.

206 From this experiment, we see that our theoretical constant c slightly overestimates how
 207 much we can increase the resolution while maintaining runtime (5% slower than the con-
 208 figuration with lower resolution). Nevertheless, with the higher resolution, we are able to
 209 reduce the RMS error by 29%. Another benefit of this strategy is the reduction in memory
 210 consumption by 21% through halving the number of bytes for each grid point, offsetting
 211 the increase in resolution.

212 4. Conclusion

213 To conclude, this project experimented how different floating-point precision and grid
 214 resolution affect the speed, performance and error in numerical modeling, using the two-
 215 dimensional Burger’s equation as the testing ground. The tested precision includes fp16,
 216 fp32, and fp64, and the resolution spans a wide range from 16×16 to 4096×4096 . A
 217 key finding is that, with higher resolution, as the wave shock develops, the imprecise

218 representation with fp16 would lead to invalid values in gradient computation, and such
 219 invalid values would propagate from the wave front to the entire simulation domain. This
 220 suggests that a compatible floating-point precision must be used for the resolution adopted.
 221 Besides, when computation resources allow, it is more advisable to increase the resolution
 222 rather than the numerical precision to achieve better results, as the improvement with
 223 resolution is much higher than that with precision. This hypothesized solution was verified
 224 by our follow up experiment, with one configuration using fp64 and 1024×1024 , and
 225 another using fp32 and 1290×1290 . While the computational times are similar, the
 226 second setup achieves an RMS error of 1.04%, which is significantly lower than that of
 227 1.46% with the first setup. Nevertheless, when the simulation time is long, it might be
 228 possible that the round-off error would build up, and higher precision would be required.
 229 This would require further testing in future work.

230 5. Bibliography

- 231 Banderier, H., Zeman, C., Leutwyler, D., Rüdüsühli, S. and Schär, C. (2024). Reduced
 232 floating-point precision in regional climate simulations: an ensemble-based statistical
 233 verification, *Geoscientific Model Development* **17**(14): 5573–5586.
- 234 Brogi, F., Bnà, S., Boga, G., Amati, G., Esposti Ongaro, T. and Cerminara, M. (2024).
 235 On floating point precision in computational fluid dynamics using openfoam, *Future*
 236 *Generation Computer Systems* **152**: 1–16.
- 237 Dagum, L. and Menon, R. (1998). Openmp: An industry-standard api for shared-memory
 238 programming, *IEEE Comput. Sci. Eng.* **5**(1): 46–55.
- 239 Gao, Q. and Zou, M. (2017). An analytical solution for two and three dimensional nonlinear
 240 burgers’ equation, *Applied Mathematical Modelling* **45**: 255–270.
- 241 Hatfield, S., Düben, P., Chantry, M., Kondo, K., Miyoshi, T. and Palmer, T. (2018).
 242 Choosing the optimal numerical precision for data assimilation in the presence of model
 243 error, *Journal of Advances in Modeling Earth Systems* **10**(9): 2177–2191.
- 244 Kimpson, T., Paxton, E. A., Chantry, M. and Palmer, T. (2023). Climate-change mod-
 245 elling at reduced floating-point precision with stochastic rounding, *Quarterly Journal of*
 246 *the Royal Meteorological Society* **149**(752): 843–855.
- 247 Lorenz, E. N. (1963). Deterministic nonperiodic flow, *Journal of Atmospheric Sciences*
 248 **20**(2): 130 – 141.
- 249 Ofenbeck, G., Steinmann, R., Caparros, V., Spampinato, D. G. and Püschel, M. (2014).
 250 Applying the roofline model, *2014 IEEE International Symposium on Performance*
 251 *Analysis of Systems and Software (ISPASS)*, pp. 76–85.
- 252 Rüdüsühli, S., Walser, A. and Fuhrer, O. (2014). Cosmo in single precision, *COSMO*
 253 *Newsletter* (14): 70–84.
- 254 Saffin, L., Hatfield, S., Düben, P. and Palmer, T. (2020). Reduced-precision parametriza-
 255 tion: lessons from an intermediate-complexity atmospheric model, *Quarterly Journal of*
 256 *the Royal Meteorological Society* **146**(729): 1590–1607.